

HPML Spring 2026 Final Exam Study Guide

- **Instructor:** Dr. Kaoutar El Maghraoui
- **Exam date:** Monday, May 11, 2026
- **Exam time:** 7:00 PM – 9:00 PM (designed for 90 minutes; you have the full 2 hours to finish)
- **Location:** Schermerhorn Hall, Room 614
- **Format:** On paper. Multiple choice, multiple-select (“select all that apply”), true/false, and short numerical problems. Question style is the same as Quizzes 1 through 6.
- **Delivery:** In-person, paper exam. Open seating — sit anywhere available in the classroom.
- **Weight:** 20% of the final course grade.

A note on scope

The topics below reflect the lectures delivered this semester and present in the lectures folder on Courseworks. Nothing outside those delivered lectures will be tested. Check Courseworks for any scope updates close to exam day.

1. Exam Rules

Closed book. No textbooks, notebooks, printouts, or lecture slides.

The exam is on paper. It is designed to take about 90 minutes; you have the full 2 hours to finish if you need them.

No electronics. Phones, tablets, laptops, smart watches, smart glasses, earbuds, and any second device must be off and out of sight (bag on the floor, not on the desk).

Calculators are allowed. Scientific or graphing calculators are fine, but anything with wireless, Wi-Fi, or Bluetooth is not. No calculator apps on phones.

You can bring one cheat sheet (format below). You must submit it with your exam.

No collaboration. The cheat sheet must be your own work.

Bring: university ID, a couple of sharpened pencils, an eraser, a calculator, and your cheat sheet.

2. Scope: What to Study vs. What to Skim

Ten lectures have been delivered. The exam does not weight them equally. Use the tiers below to budget your time.

Tier 1: heavy coverage (~70% of the exam)

These are the priority topics.

Lecture	Topic	Source file
L3	PyTorch Performance and Profiling	HPML-03-Pytorch-performance-Spring2026.pdf
L4	CUDA Basics	HPML-04-CUDA-basics-Columbia-Spring2026.pdf
L5	Advanced CUDA	HPML-05-CUDA-Advanced_Spring2026.pdf
L6	Distributed Deep Learning (DDL)	HPML-6-DDL-Spring2026-columbia.pdf
L9	Efficient Transformers and FlashAttention	HPML9_EfficientTransformers_FlashAttention.pdf

Tier 2: light coverage (~25% of the exam)

One or two conceptual questions each. Know the definitions and one key tradeoff.

- L1: Intro to HPC and AI (HPC drivers, scaling definitions, Amdahl's law)
- L2: Performance optimization methodology and PyTorch fundamentals (autograd, optimizers)
- L7: Quantization (PTQ vs. QAT, INT8/FP16/FP8, dynamic vs. static)
- L8: Sparsity and Pruning (structured vs. unstructured)
- L10: Knowledge Distillation (teacher/student, soft vs. hard targets)

Anything not on this list is out of scope.

3. Tier 1 Study Checklist

Use this as a self-check. If you cannot answer any bullet in one sentence, re-read that slide.

3.1 PyTorch Performance and Profiling (L3)

- Why Python is slow: dynamic typing, reference counting, the GIL, PyObject overhead, bytecode interpreter.
- Static vs. dynamically typed languages. Which is C++? Python? JavaScript?
- Imperative (eager) vs. declarative (graph) computation-graph construction, and which allows more optimization.
- `.pyc` files, the Python VM, peephole optimization.
- The PyTorch 2 compile stack: TorchDynamo (captures bytecode, produces an FX graph), AOTAutograd, TorchInductor (Triton backend on GPU, C++/OpenMP on CPU).
- What guards are in TorchDynamo and why they matter for cache reuse.
- DataLoader features: batching, shuffling, parallel workers, pinned memory, prefetching. And what it does not do (it never updates weights).
- Profiling tools: `torch.profiler`, `nvprof` / Nsight Systems, timeline traces.

3.2 CUDA Basics (L4)

- Grid, block, thread hierarchy. How `blockIdx`, `threadIdx`, `blockDim`, and `gridDim` combine to index data.

- Warp = 32 threads. SIMT execution, and the cost of warp divergence.
- Host vs. device code. The `__global__`, `__device__`, and `__host__` qualifiers.
- Kernel launch syntax `kernel<<<grid, block>>>(args)`, and how to pick grid and block size for a given N.
- AoT vs. JIT compilation. PTX (portable IR) vs. SASS (device binary).
- What compute capability controls.
- Heterogeneous memory: `cudaMalloc`, `cudaMemcpy` (H2D, D2H, D2D), async copies.
- A minimal vector-add kernel. Be able to read one and trace it.

3.3 Advanced CUDA (L5)

- Memory hierarchy from fastest to slowest: registers, shared memory, L1, L2, global (HBM).
- Memory coalescing: consecutive threads should hit consecutive addresses. Stride-1 is best.
- Shared memory: per-block, roughly 100 times faster than global, 32 banks, watch for bank conflicts.
- Tiled matrix multiplication. Why tiling cuts global memory traffic, and how tile size ties to the shared-memory footprint.
- `__syncthreads()` : when you need it, when you don't (intra-warp sync is implicit).
- Occupancy = active warps per SM divided by max warps per SM. Higher occupancy helps hide latency.
- CUDA streams and overlapping compute with H2D/D2H transfers.
- Unified Virtual Memory (UVM). Convenient, but watch the page-fault cost.
- cuDNN: forward and backward conv descriptors, algorithm selection (`FindAlgorithm`).

3.4 Distributed Deep Learning (L6)

- Data, model, tensor, pipeline, and hybrid parallelism. One sentence each.
- Synchronous vs. asynchronous SGD. Stragglers and stale gradients.
- Centralized (parameter server) vs. decentralized (AllReduce).
- Collective primitives: Broadcast, Reduce, AllReduce, Reduce-Scatter, AllGather, AllToAll. Know what each one does.
- Ring AllReduce. Bandwidth-optimal. Each rank sends $(N-1)/N$ of the data in each of $2(N-1)$ steps.
- NCCL (GPU, topology-aware), Gloo (CPU fallback), MPI, CUDA-aware MPI, GPUDirect RDMA, NVLink/NVSwitch, InfiniBand.
- Strong vs. weak scaling. Know the definitions from Quiz 1 and be able to recognize examples.
- Overlapping the backward pass with gradient AllReduce, and why it is critical for scaling.
- PyTorch DDP vs. FSDP (sharded parameters, gradients, and optimizer states).

3.5 FlashAttention and Efficient Transformers (L9)

- Standard attention: Q, K, V; $\text{softmax}(QK^T/\sqrt{d_k})V$; $O(N^2)$ time and memory.
- Why standard attention is memory-bound, not compute-bound, on modern GPUs. Softmax, mask, and dropout all have low arithmetic intensity.
- HBM vs. SRAM. HBM is large and slow; SRAM is small and roughly 10 times faster.
- The three FlashAttention ideas:
 1. Tiling. Split Q, K, V into blocks that fit in SRAM.

2. Online softmax. Track a running max m and normalizer ℓ per tile so softmax can be computed incrementally, without materializing the full $N \times N$ matrix.
 3. Backward recomputation. Don't store the attention matrix. Recompute from Q, K, V in SRAM.
- Result: same $O(N^2)$ FLOPs, but $O(N)$ HBM I/O, 2 to 6 times wall-clock speedup, and longer context lengths become practical.
 - Kernel fusion. Why fusing softmax, matmul, mask, and dropout into one kernel wins.
 - Encoder vs. decoder attention. Causal mask.

4. Tier 2 Study Checklist (quick hits)

- **HPC and Amdahl.** Speedup = $1 / ((1-p) + p/s)$. Harmonic mean for averaging rates, arithmetic mean for averaging times, geometric mean for speedups. See Quiz 1.
- **Roofline model.** Arithmetic intensity (FLOPs/byte) on the x-axis, attainable FLOPs/s on the y-axis. Memory-bound to the left of the ridge, compute-bound to the right.
- **Quantization.** FP32 $\rightarrow$ FP16/BF16 $\rightarrow$ INT8 $\rightarrow$ INT4 $\rightarrow$ FP8. PTQ (no retraining) vs. QAT (simulated quant during training). Static (offline calibration) vs. dynamic (per-inference).
- **Pruning.** Unstructured (fine-grained, hard to accelerate) vs. structured (channel, head, or layer; hardware-friendly). Magnitude vs. movement pruning. 2:4 structured sparsity on Ampere and newer.
- **Knowledge distillation.** Teacher logits softened by a temperature T, KL divergence loss, and a combination of soft and hard targets.

5. Numbers Worth Memorizing for Your Cheat Sheet

Fact	Value
Warp size	32 threads
Max threads per block (typical)	1024
Shared-memory banks	32, 4-byte granule
Common block sizes	128, 256, 512
HBM latency	about 400 to 600 cycles
FP32 to FP16 memory reduction	2×
FP32 to INT8 memory reduction	4×
Standard attention memory	$O(N^2)$
FlashAttention HBM I/O	$O(N^2 \cdot d^2 / M)$, where M is SRAM size
Amdahl's speedup	$S = 1 / ((1-p) + p/s)$

Sample practice questions are collected in the Appendix at the end of this guide.

6. Cheat Sheet: Format and Rules

Required format

- One sheet, US Letter (8.5" × 11"). Both sides allowed, so effectively two pages.
- Handwritten or typed. Both are fine. Minimum font size 8 pt if typed.
- No fold-outs, no attached notes, no tabs, no photocopies of slides.
- Must be your own work. Copying another student's sheet is an academic-integrity violation.
- Write your name and UNI in the top-right corner of the front side.
- Bring the original. No photocopies of another student's sheet.
- Loose single sheet. Nothing stapled or paperclipped to it.
- Turn it in with your exam at the end. It will be stapled to your exam.

Suggested contents

Put things on your sheet that you cannot derive in 30 seconds under pressure. Skip anything a calculator handles.

1. **Formulas.** Amdahl's law, harmonic mean, roofline intersection point, the attention formula $\text{softmax}(\mathbf{QK}^T/\sqrt{d_k})\mathbf{V}$, and FlashAttention I/O complexity.
2. **Constants.** Warp = 32, common block sizes, 32 shared-memory banks, and byte sizes for FP32/FP16/INT8.
3. **Tables.**
 - Collective primitives (Broadcast, Reduce, AllReduce, Reduce-Scatter, AllGather), one line each.
 - CUDA memory hierarchy: scope, latency, size, typical use.
 - PyTorch 2 stack: Dynamo, AOTAutograd, Inductor, Triton, one line each.
 - PTQ vs. QAT, and static vs. dynamic quantization.
4. **Small diagrams.** Ring AllReduce. Tiled matmul with shared memory. FlashAttention tiling (Q-tiles against K/V-tiles with running m and ℓ).
5. **Gotchas.** Things you keep forgetting. For example: `__syncthreads` is block-scope only, warp divergence has a real cost, and which mean (harmonic / arithmetic / geometric) applies where.

How to prepare it

Build the sheet while you study, not after. Deciding what goes on it is most of the work.

Resist the urge to cram. On the exam, the real bottleneck is time, not recall, and a dense wall of text is hard to search when you are stressed. Leave white space, use columns, group by topic.

Do two or three practice runs with only your sheet before the exam. If you keep reaching for something that isn't on it, add it.

7. Suggested Study Plan (last 10 days)

Day	Focus
-10 to -8	Re-read L4 and L5 slides. Work through a tiled matmul by hand. Write the CUDA section of your cheat sheet.
-7 to -6	L3 (PyTorch perf) and L2 (profiling). Re-derive why Python is slow. Practice telling eager and graph mode apart.
-5 to -4	L6 (DDL). Draw ring AllReduce on paper. Know the difference between data, model, tensor, and pipeline parallelism.
-3 to -2	L9 (FlashAttention). Draw the tiling diagram. Explain online softmax out loud.
-1	Take the Appendix practice questions, timed. Finalize your cheat sheet. Review Tier 2 (quantization, pruning, distillation).
Exam day	Arrive early with your university ID, a sharpened pencil, an eraser, a calculator, and your cheat sheet.

Good luck.

Appendix: Sample Practice Questions

Twenty practice questions in the same style as the real exam. The real exam will have roughly 25 to 35 questions drawn from the same topics with different wording. Answers are at the end of the appendix.

A.1 Multiple Choice

Q1. A CUDA warp consists of how many threads executing in lockstep (SIMT)?

- A. 16
- B. 32
- C. 64
- D. 128

Q2. In a tiled matrix multiplication kernel, the main reason to load tiles of A and B into shared memory is:

- A. Shared memory has more capacity than global memory.
- B. Shared memory data persists across kernel launches.
- C. Each element loaded from global memory is reused by many threads within the block, amortizing the HBM cost.
- D. Shared memory does not require `__syncthreads()`.

Q3. Which statement best describes FlashAttention?

- A. It changes attention complexity from $O(N^2)$ to $O(N \log N)$ FLOPs.

- B. It performs the same $O(N^2)$ FLOPs as standard attention but reduces HBM I/O by tiling and online softmax, keeping intermediates in SRAM.
- C. It replaces softmax with a linear approximation.
- D. It shards the attention matrix across GPUs.

Q4. Which collective is bandwidth-optimal for synchronizing gradients in data-parallel training and is the algorithm used by NCCL on a uniform ring topology?

- A. Parameter-server push/pull
- B. Ring AllReduce
- C. AllToAll
- D. Broadcast

Q5. The decode phase of autoregressive LLM inference is typically:

- A. Compute-bound, because each step does a large matmul.
- B. Memory-bandwidth-bound, because each generated token requires reading all model weights once.
- C. Network-bound, because of NCCL traffic.
- D. I/O-bound, because of dataloader prefetching.

Q6. In PyTorch 2, the component that hooks into CPython's frame evaluation API to capture Python bytecode and produce an FX graph is:

- A. TorchScript
- B. TorchInductor
- C. TorchDynamo
- D. AOTAutograd

Q7. On an NVIDIA GPU, which memory has the lowest access latency for a thread?

- A. Global memory (HBM)
- B. L2 cache
- C. Shared memory
- D. Registers

Q8. You train a model with DDP on 8 GPUs. Per-GPU batch = 32, so global batch = 256. You then move to 16 GPUs with per-GPU batch still = 32 (global batch = 512), and runtime per epoch is roughly unchanged while total work doubles. This is an example of:

- A. Strong scaling
- B. Weak scaling
- C. Amdahl saturation
- D. Gustafson's law failing

Q9. Which of the following is a property of the imperative (eager) approach to computation graphs?

- A. The full graph must be declared before execution.
- B. Operations are executed immediately as Python statements run.
- C. It always produces faster code than graph mode.
- D. It prevents the user from using Python control flow.

A.2 Select All That Apply

Q10. Which are characteristics of shared memory on an NVIDIA GPU? (Select all that apply.)

- A. Scoped to a single thread block.
- B. Persists across kernel launches.
- C. Divided into banks; simultaneous accesses to the same bank by different threads in a warp cause conflicts.
- D. Faster than global memory.

Q11. Which of the following are optimizations provided by TorchInductor / `torch.compile`? (Select all that apply.)

- A. Kernel fusion to reduce memory traffic.
- B. Triton code generation for GPU kernels.
- C. Automatic distributed training across nodes.
- D. Graph-level dead code elimination.

A.3 True/False

Q12. FlashAttention reduces the asymptotic FLOP count of attention below $O(N^2)$.

- True
- False

Q13. In synchronous data-parallel SGD, a slow straggler worker can stall the entire step because all workers must complete their backward pass before AllReduce finishes.

- True
- False

A.4 Code and Profiling Interpretation

Q14. A student launches this CUDA kernel for $N = 100,000$:

```
__global__ void vec_add(float* a, float* b, float* c, int N) {  
    int i = threadIdx.x;  
    if (i < N) c[i] = a[i] + b[i];  
}  
// launch:  
vec_add<<<256, 256>>>(a, b, c, N);
```

The output is wrong. Only a small prefix of `c` is correct. What is the bug?

- A. The kernel should use `blockDim.x` instead of `threadIdx.x`.
- B. The thread index ignores `blockIdx.x` and `blockDim.x`, so every block recomputes the same 256 elements and writes the same positions.
- C. The launch is missing `__syncthreads()` after the write.
- D. The `if (i < N)` guard is unnecessary and causes undefined behavior.

Q15. Consider two access patterns inside a warp of 32 threads (each thread reads one `float`):

```
// Pattern A
float x = A[threadIdx.x];
// Pattern B
float x = A[threadIdx.x * 32];
```

Which is true?

- A. Both coalesce into a single 128-byte transaction.
- B. A coalesces into one transaction; B issues 32 separate transactions and wastes most of the bandwidth.
- C. B coalesces better than A because of stride alignment.
- D. Neither pattern can be coalesced on modern GPUs.

Q16. A PyTorch profiler trace for one training iteration shows:

Stage	Time
DataLoader next()	420 ms
forward	38 ms
backward	55 ms
optimizer.step	6 ms
GPU idle (waiting on host)	410 ms

What is the most likely bottleneck and the right first fix?

- A. Forward pass is too slow; enable `torch.compile`.
- B. Optimizer is slow; switch from Adam to SGD.
- C. The input pipeline is the bottleneck; increase `num_workers`, enable `pin_memory=True`, and prefetch.
- D. The GPU is compute-bound; upgrade the GPU.

Q17. A team benchmarks DDP on 1, 2, 4, and 8 GPUs with a fixed global batch size. Time per epoch:

GPUs	Time (s)
1	400
2	210
4	115
8	70

Which statement is correct?

- A. This is perfect linear strong scaling.
- B. This is strong scaling, and efficiency at 8 GPUs is about 71% ($400 / (8 \times 70)$).
- C. This is weak scaling because the batch size is fixed.
- D. Efficiency improves with more GPUs because of AllReduce overlap.

Q18. `nvidia-smi dmon` during training shows GPU utilization at 12% and memory utilization at 3%, while CPU cores are pegged near 100%. The most likely explanation is:

- A. The model is too large for the GPU and is paging.
- B. The CPU input pipeline cannot feed the GPU fast enough; the GPU is idle most of the time.
- C. NCCL is saturating the interconnect.
- D. The optimizer is running on the CPU instead of the GPU.

Q19. A student writes attention this way in PyTorch:

```
scores = Q @ K.transpose(-2, -1) / math.sqrt(d)
attn    = torch.softmax(scores, dim=-1)
out     = attn @ V
```

For very long sequences on a single GPU, the program runs out of memory. The main reason is:

- A. `softmax` allocates gradient buffers per token.
- B. `Q @ K.T` materializes a dense $N \times N$ matrix in HBM, and its memory scales with N^2 .
- C. The `@` operator cannot be fused on GPU.
- D. `math.sqrt` is run on CPU and stalls the pipeline.

Q20. A student reports `torch.cuda.synchronize()` timings for the same matmul, run in a Python for-loop of 100 iterations:

- Version A: 12.0 seconds total
- Version B: 0.18 seconds total

Version A calls `.item()` on the result inside the loop. Version B does not. The most likely explanation is:

- A. `.item()` runs the matmul on CPU.
- B. `.item()` forces a host-device synchronization every iteration, serializing launches that would otherwise be pipelined asynchronously on the GPU.
- C. `.item()` allocates a new tensor every call.
- D. The matmul in Version B was cached by cuDNN.

A.5 Answer Key

- **Q1:** B
- **Q2:** C
- **Q3:** B
- **Q4:** B
- **Q5:** B
- **Q6:** C
- **Q7:** D
- **Q8:** B
- **Q9:** B
- **Q10:** A, C, D
- **Q11:** A, B, D
- **Q12:** False (same $O(N^2)$ FLOPs; only the I/O is reduced)
- **Q13:** True

- **Q14:** B. The global index must be `blockIdx.x * blockDim.x + threadIdx.x`.
- **Q15:** B. Stride-1 coalesces; stride-32 does not.
- **Q16:** C. The GPU sits idle waiting on the host; fix the input pipeline.
- **Q17:** B. Strong scaling; efficiency = $T_1 / (P \cdot T_p) = 400 / 560 \approx 0.71$.
- **Q18:** B. Classic CPU-bound dataloader symptom.
- **Q19:** B. The $N \times N$ attention matrix is the memory blow-up.
- **Q20:** B. `.item()` forces a blocking sync and kills async GPU launches.